Portfolio Project - Team Formation, Brainstorming and MVP

This outline details the Team Formation and Idea Development process. It is organized to guide the submission of deliverables for the initial project phase, specifically focusing on team establishment, idea generation, and the final selection and definition of the Minimum Viable Product (MVP) concept.

1. Team Formation Overview

Team Establishment and Collaboration:

This section outlines the formation of the project team, covering individual roles, relevant backgrounds, and the strategies for effective collaboration.

1.1. Assigned Roles

The table below details the team's role assignments, primary technical roles designated for the entire project. A rationale for each assignment is also provided.

Role	Assignee	Justification (Why assigned)
Project Manager	Mickael MUR	Background in management and planning.
Technical Roles		
Backend Developer	Jordann Miso	Experience in Python/ Databases/API
Frontend Developer	Mickael MUR	Skills in React/Mobile development.
UI/UX Designer	Mickael MUR Jordann Miso	Focus on user experience and design principles.
QA Specialist	Jordann Miso Mickael MUR	Attention to detail and testing methodologies.

1.2. Communication & Collaboration Norms

- **Primary Communication Tool:** Discord
  - → Expected response time, use of dedicated channels

- **Documentation Tool:** Google Drive
    → File Structure and Naming Conventions
- **Meeting Frequency & Type:** Daily 15-minute stand-ups, Weekly reports
- **Project management:** Trello
    → Project management and deadline.

1.3. Stakeholders

The key stakeholders, their roles, and their impact on the project's success are listed here.

Stakeholder	Role and Description	Impact (Influence and Interest)
Rémi Marcais	Technical Advisor/Mentor	High Influence: Validates technical architecture and project milestones.
Holberton Students	The ultimate end-users of the product.	High Interest/High Impact: Their feedback is critical to MVP success.

2. Ideas Explored

This section documents the initial brainstorming process, the criteria used for evaluation, and the rationale for selecting the final MVP concept.

2.1. List of Brainstormed Ideas

The following table summarizes the ideas generated during the brainstorming phase.

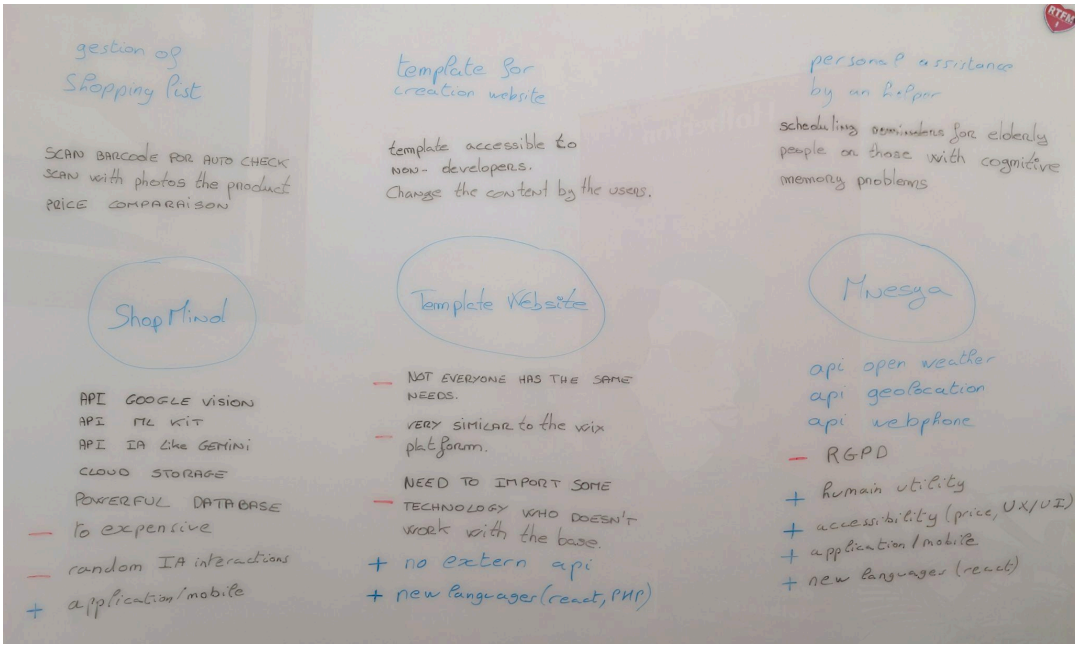
Idea Name	Short Description	Feasibility and Risk
ShopMind Application	Application mobile for making a shop list and automatically check the article with scan barcode or take the article on photo.	Project cost too high (External API and AI Hosting)
Template Web Site	Creation of a universal website for artisans and commerce who can make himself the content.	The Wix platform does this job very well, not a very innovative idea.

Idea Name	Short Description	Feasibility and Risk
Mnesya Application	Application mobile for helping the isolated person or with cognition problems of memory	Simple MVP using Firebase for notifications. Minimal external dependencies. Simple interface for elderly people, focusing on core reminder functionality.

2.2. Reasons for Idea Selection/Rejection

- **Rejected Idea 1:** ShopMind
  - → Reason for Rejection: Idea interesting but not useful in the world, maybe in the future simply a gadget and after research the application must be too expensive with IA hebergement and request API.

- **Rejected Idea 2:** Template Web Site
  - → Reason for Rejection: Good potential, but we have ever Wix making this project greatly and we have more websites for creation templates for beginners with IA.



- **Selected Idea :** Mnesya

2.3. Brainstorming Techniques Used

To organize the overall structure of the Mnesya project, we used the **Mind Mapping** technique.
we first placed the main concept at the center:

- **"Mnesya – Elderly Care Assistance Application."**
  From this central point, we created several branches corresponding to the main components of the project:
- **User Interfaces:** a simplified interface for the elderly user and an advanced interface for the caregiver.
- **Main Modules:** reminder management, notifications, real-time tracking, weather, alerts.
- **Technical Architecture:** FastAPI backend, React Native frontend, PostgreSQL database, worker, external APIs.
- **Security:** JWT, PIN, permissions, encryption, logs.
- **Database:** users, profiles, reminders, statuses, alerts, voice notes.
- **Development Roadmap:** phased development, priorities, future features.
- **Deployment and Infrastructure:** Docker, NGINX, VPS/Cloud.

This mind map allowed us to clearly visualize the different parts of the project, define the relationships between modules, and most importantly, organize the documentation into coherent sections.
It also served as a foundation for defining:

- The Backend structure,
- The Frontend screens,
- The separation of responsibilities,
- The development timeline.

For the Mnesya project, we applied the **SCAMPER** technique to explore ways to improve and innovate the application's features.
We analyzed the main functionalities of the app (user interface, reminder system, notifications, tracking, and alerts) through each SCAMPER lens:

- **Substitute:** Replacing complex menus with ultra-simplified interfaces and large buttons for elderly users.
- **Combine:** Integrating reminders with voice playback and push notifications to enhance usability.
- **Adapt:** Adapting features from existing apps to fit the specific needs of elderly people and caregivers.
- **Modify:** Modifying the notification system to allow repeated reminders every 5 minutes.
- **Put to another use:** Using the voice module both for reminders and for user voice notes/journal.
- **Eliminate:** Removing unnecessary steps and interactions to reduce cognitive load.

- **Reverse / Rearrange:** Reorganizing the app workflow so caregivers manage profiles and reminders centrally while users receive only essential interactions.

For the Mnesya project, we applied the **"How Might We" (HMW)** method to generate solutions and ideas for improving the user experience and addressing challenges faced by elderly users and their caregivers.

I formulated several **HMW questions** based on the project's goals and constraints:

- *"Makes reminders easy and accessible for elderly users with cognitive difficulties?"*
- *"Allows caregivers to efficiently manage multiple profiles and track tasks in real time?"*
- *"Integrates voice notifications and TTS to improve accessibility and reduce errors?"*
- *"Simplify the interface while keeping all essential functionalities for both users and managers?"*
- *"Provides alerts and notifications that are timely, clear, and actionable?"*

Answering these questions guided the design of the app's features, such as the **ultra-simplified interface, programming reminders, push notifications, and real-time tracking**.

2.4. Evaluation Criteria Defined

The criteria used to objectively assess and rank the viability of each brainstormed idea:

Feasibility (Technical/Time)

Technical Feasibility:

- The backend will be developed with **FastAPI (Python)**. We had prior experience with Python through school projects and personal repositories, which provides a solid foundation for building APIs and server logic.

- The frontend will be developed with React Native for Android. While we have limited experience with React Native, following tutorials, using pre-built components, and consulting the official documentation makes this achievable.

- PostgreSQL database management and integration of Firebase for push notifications will require learning but remain manageable thanks to the modular architecture and available examples.

- Deployment using **Docker and Docker Compose** simplifies environment setup and ensures consistency across platforms.

Time Feasibility:

- The project is structured in phases: core functionality → security & permissions → advanced services. This phased approach allows us to focus on essential features first.
- The use of **modular architecture and reusable components** saves time and reduces errors.
- With targeted learning and good organization, we can deliver a **functional MVP** within the project timeline.

Impact (User Value): How much value does this solution provide to the target user?
Technical Alignment: Does this project allow the team to utilize or learn desired technologies?
Market Opportunity: Does this idea fill a gap or offer a clear advantage over existing solutions?

3. Selected MVP Concept

This section provides the complete definition of the chosen Minimum Viable Product (MVP).

3.1. Summary & Rationale

- **MVP Title:** Mnesya
- **Core Problem Solved:** Helping elderly people with memory loss remember essential daily tasks while giving caregivers peace of mind through simple tracking.
- **Proposed Solution:** A mobile app with a simplified interface for seniors (visual push notifications with three-button responses) and a caregiver dashboard for remote task scheduling and simple monitoring.

3.2. The Problem & Solution Refined

Detailed Problem Explanation:

Elderly people with memory problems like Alzheimer's or dementia often forget important daily tasks such as taking their medication, eating meals, or going to appointments. This can lead to serious health issues. Current solutions like sticky notes, basic phone alarms, or family members calling multiple times a day don't work well because they're either too complicated, easy to ignore, or annoying.

At the same time, family caregivers and professional helpers are constantly worried and stressed because they can't know if their elderly relatives or patients actually did what they were supposed to do without physically being there or calling them all the time.

This creates a situation where elderly people lose their independence and caregivers get exhausted from the constant anxiety and need to check in. The problem gets even worse when one caregiver needs to look after multiple elderly people it becomes nearly impossible to keep track of who did what and when.

Detailed Solution Explanation:
Mnesya addresses this problem with a mobile app that has two different interfaces for the two types of users. For elderly users, the interface is extremely simple: it has big buttons that are easy to see and tap, push notifications with text reminders, and just three buttons to respond (Done, Remind Me Later, or I Can't Do This).

For caregivers, there's a management dashboard where they can create and schedule one-shot reminders with title and description, manage multiple profiles if they're caring for several people, and see in real-time whether each reminder was completed or not.

The app keeps a simple history of all reminders with timestamps and responses, allowing caregivers to track which tasks were completed, postponed, or refused. This MVP focuses on the core functionality needed to prove the concept works.

Advanced features like voice notifications, automatic alerts, recurring reminders, weather integration, and detailed analytics are planned for future versions once the core system is validated with real users.

3.3. Target Audience / Users

Primary User Persona:

- **Technical :** Low to very low. Struggles with complex smartphones, prefers simple interfaces with large buttons. May have mild cognitive impairments (early Alzheimer's, memory loss) or elderly persons.
- **Environment:** Lives alone or in assisted living. Owns an Android smartphone (often provided by family). Needs reminders for mediations, meals, medical appointments, and family calls.
- **Needs:**
    - → Ultra-simplified interface without complex menus
    - → Clear and large visual notifications
    - → One-tap response (3 big buttons)
    - → No configuration required (everything managed by caregiver)
    - → Preserved autonomy and dignity

Secondary User Persona:

- **Technical:** Medium to high. Comfortable with smartphones and applications. Can be a family member (daughter, son) or professional caregiver.
- **Environment:** Balances professional life with caring for one or more elderly relatives. Constant stress from not knowing if essential tasks are being completed. uses smartphones regularly throughout the day.
- **Needs:**
    - → Quickly create and schedule reminders
    - → Manage multiple profiles from a single account
    - → See real-time task completion status
    - → Simple history tracking for adherence monitoring
    - → Reduce anxiety without constantly calling
    - → Reliable and secure solution

3.4. Application Type & Details

Mobile Application (Android for MVP) with backend API Server

- **Primary Interface:** Native mobile app for both elderly users and caregivers.
- **Supporting Components:**

    - → Push Notification system for real-time reminders.
    - → Backend REST API for data management and synchronization.
    - → Worker service for scheduled task execution.

Key Technology Stacks:

Frontend (Mobile App):

- **Framework:** React Native (cross-platform Android development for MVP)
- **Navigation:** React Navigation
- **Push Notification:** Firebase Cloud Messaging (FCM)

Backend (API Server):

- **Framework:** FastAPI (Python) - high-performance REST API
- **Database:** PostgreSQL - relational database for structured data (the CAP theorem)
- **Authentication:** JWT (JSON Web Tokens) for secure sessions
- **Task Scheduling**: APScheduler - simple scheduler for one-shot reminders

External Services:

- **Push Notifications:** Firebase Cloud Messaging

Infrastructure:

- **Containerization:** Docker & Docker Compose

- **Reverse Proxy:** NGINX
- **Deployment:** VPS

Development Tools:

- **API Documentation:** Swagger/OpenAPI (auto-generated by FastAPI)
- **Database Migrations:** Alembic
- **Version Control:** Git

3.5. Reasons for Selection

Alignment with Feasibility Criteria

The project is academically viable due to its manageable scope and the use of a robust, well-supported technology stack (React Native, FastAPI, PostgreSQL).

Key Factors for Feasibility:

- **Technology & Support:** Relies on established, open-source frameworks with extensive community documentation.
- **Implementation Strategy:** Complex custom development is minimized by leveraging existing libraries for core features (push notifications, database operations, authentication).
- **Development Plan:** The project is designed for incremental development, allowing the team to start with basic functionality (reminders, simple interfaces) and progressively add advanced features (recurrence, analytics, voice notifications).
- **Consistency & Resources:** Docker guarantees a uniform development environment. Firebase provides free tiers suitable for MVP development.
- **Scope:** The scope is appropriate for a single semester while offering clear pathways for future expansion (voice notifications, weather integration, advanced analytics).

High Potential for User Impact and Value

Project Justification and Impact

This project addresses a real and growing problem that affects over 55 million people worldwide living with dementia. The large target audience makes this solution immediately relevant and useful. What makes it even more practical is that team members could actually use it with their own elderly family members, which would give us real feedback from actual users during development.

Solution and Benefits

The dual-interface design offers simultaneous benefits: enhanced independence and safety for elderly users, and reduced anxiety for their caregivers. The underlying problem is highly relatable and emotionally compelling, making it an excellent feature for professional portfolios and interviews.

Success and Future Potential

Success can be clearly measured using metrics like reminder completion rates and medication adherence. Beyond immediate application, the project has strong potential for deployment as a non-profit or social entrepreneurship venture.

Opportunities for Technical Skill Development Within the Team

This project offers broad full-stack learning opportunities, creating a robust portfolio piece.

Technical Skill Development:

- Cross-Platform Mobile: Utilizing React Native for mobile development.
- Backend Engineering (FastAPI): Gaining expertise in RESTful API design, database modeling, JWT authentication, and task scheduling.
- DevOps: Essential experience with Docker for modern deployment practices.
- Service Integration: Learning service orchestration and error handling through integration with third-party APIs (Firebase).

Marketable Deliverables:

- Comprehensive Portfolio: Demonstrates full-stack capabilities, third-party integrations, and a user-centered approach.
- User-Centered Design: Focus on accessibility-driven UX/UI design and adherence to strict security protocols for protecting vulnerable user data.

These combined elements align precisely with the requirements sought by employers in junior developer candidates.

3.6. Key Features & SMART Objectives

Key Features (The "What")

Feature 1: Dual Authentication System

- **Manager Authentication:** Secure JWT-based login with email/password for caregivers to access the management dashboard

- **User Authentication:** Simple 4-6 digit PIN code for elderly users to access their simplified interface
- **Multi-profile Management:** Managers can create and oversee multiple elderly user profiles from a single account
- **Role-based Access:** Clear separation between manager capabilities (full control) and user capabilities (view-only reminders)

Feature 2: One-Shot Reminder System

- **Reminder Creation:** Managers can schedule one-shot reminders with title, description, and date/time
- **Push Notifications:** Automatic notifications sent to elderly users at scheduled times
- **Three-Button Response:** Large, accessible buttons for users to respond: Done, Remind Later, Unable
- **Simple Tracking**: Managers can see which reminders were completed, postponed, or refused

Feature 3: Simple Tracking Dashboard

- **Status Monitoring:** Managers see updates when reminders are completed, postponed, or refused
- **History View:** Simple log of all reminders with timestamps and responses
- **Profile Overview:** Dashboard displaying all managed profiles with their reminders

SMART Objectives (The "Why" and "How Measured")

Objective 1: Develop a Fully Functional Minimum Viable Product (MVP)

Specific: Create a working mobile application with two parts: one for the elderly person (the user) and one for the caregiver (the manager). It must include basic functions: logging in, creating and scheduling one-shot reminders, sending push notifications, a simple three-button response system, and a basic dashboard to track activities.

Measurable:

- Complete 100% of essential user tasks (approximately 15 to 18 tasks).
- Launch the API with all 13 essential endpoints (3 auth + 3 profiles + 6 reminders + 1 history).
- Deploy the mobile application so it can be tested on Android phones.
- Successfully pass the full notification chain test (create → send → receive → respond → log tracking).

Achievable: By using existing and well-documented development tools (such as React Native and FastAPI), and common services (like Firebase for notifications), the team can build these functions with their current skills and within the allotted time.

Relevant: An MVP proves that our idea works. It demonstrates the application's value to users and stakeholders, serves as a portfolio piece of our work, and forms the basis for adding further improvements later.

Time-Bound: Complete development, testing, and deployment within 7 weeks, with regular weekly work milestones and progress review checkpoints.

3.7. Project Scope

In-Scope Deliverables	Out-of-Scope Deliverables (V2/Future)
Core Authentication & User Management - Dual authentication system (JWT for managers, PIN for elderly users) - User registration and login for managers - Profile creation and management (managers can create multiple elderly user profiles) - Basic role-based access control	**Advanced Reminder Features** - Custom complex recurrence patterns (e.g., "first Monday of every month") - Conditional reminders based on weather or other triggers - Voice recording for custom audio messages by managers - Image attachments for reminders - Location-based reminders - Text-to-Speech for reading reminders aloud - Edit and delete reminders - Recurring reminders (daily, weekly, custom patterns) - Image and audio attachments - 5-minute auto re-reminder functionality - Custom notification sounds and vibrations - Scheduled notification batching - Alert System - Automatic alerts for managers when reminders are ignored - Escalation rules for critical reminders - Multi-channel alerts (email, SMS) - Weather - Display current weather conditions - Weather-based reminder suggestions - Severe weather alerts

In-Scope Deliverables	Out-of-Scope Deliverables (V2/Future)
Reminder System (Core Features) - Create one-shot reminders with title, description, date/time - Schedule reminders for specific profiles - View reminder details	**AI & Machine Learning Integration** - Predictive analytics for identifying patterns in missed reminders - AI-powered personalized reminder timing optimization - Natural language processing for voice commands - Sentiment analysis of user responses
Notification & Response System - Push notifications via Firebase Cloud Messaging - Three-button response interface (Done, Remind Later, Unable)	**Advanced Tracking & Analytics** - Detailed graphs and visualizations (line charts, pie charts, trends) - Exportable reports (PDF, CSV) - Comparative analytics across multiple profiles - Predictive alerts based on historical patterns - Statistics and completion rate analytics - Alert system for managers when reminders are missed or refused - Upcoming and overdue reminders dashboard
Tracking & Monitoring - Real-time reminder status updates (pending, completed, postponed, refused, ignored) - Simple history view of all reminders with timestamps and responses - List view of reminders by profile	**Communication Features** - In-app messaging between managers and users - Video call integration - Emergency contact system with one-tap calling - Group management for multiple caregivers per profile
Basic User Interface - Simplified elderly user interface with large buttons and high contrast - Manager dashboard with essential CRUD operations - Functional but minimal design (focus on usability over aesthetics)	**Polished UI/UX** - Custom animations and transitions - Dark mode support - Personalized themes and color schemes - Advanced accessibility features (screen reader optimization, haptic feedback) - Onboarding tutorials and tooltips

In-Scope Deliverables	Out-of-Scope Deliverables (V2/Future)
	- Home screen widget displaying next reminder and weather
Technical Infrastructure - FastAPI backend with PostgreSQL database - React Native mobile app (iOS and Android) - Docker containerization for development and deployment - Basic API documentation with Swagger - Database migrations with Alembic - APScheduler for scheduled tasks	**Multi-Platform & Integration** - Web dashboard for managers (in addition to mobile) - Desktop applications (Windows, macOS) - Smart home integration (Alexa, Google Home) - Wearable device support (Apple Watch, Fitbit) - Integration with electronic health records (EHR) - iOS deployment
	Advanced Technical Features - Real-time sync with WebSockets - Offline mode with local data caching - Multi-language support (internationalization) - Two-factor authentication (2FA) - HIPAA compliance certification - Advanced load balancing and scalability optimization
	Business Features - Subscription/payment system for premium features - Multi-tenancy for healthcare organizations - Admin panel for system monitoring - User analytics and behavior tracking (with privacy compliance)

3.8. Identified Risks & Mitigation Strategies

Risk Type	Potential Risk	Mitigation Strategy
Technical	External API doesn't work	Allocate more time for this part and have an emergency list
Technical	Can't be Deployed on iOS without a Mac	Only deploy the application on Android for the MVP
Time management	Don't have the time to do all tasks	Gestion of the time management with deadline with Trello
Test Management	Difficulty to get feedback from other user	Simplify the app download for test with more users who have an android phone
Infrastructure	Server crashes right before our presentation or demo	Use Docker so we can run it anywhere. Keep a backup on our laptops, use a reliable hosting service like VPS that doesn't go down often.
Organizational	Some team members don't know React native, Firebase or FastApi yet	Spend the first week doing tutorials together, pair up experienced and beginner members for coding, write down what we learn so everyone can reference it